



PANDAS

Autores:

**Sheyla Tumbaco
Samira Sánchez**



Pandas-Data frame

¿Qué es Pandas?

Pandas es una biblioteca de Python diseñada para la manipulación, organización y análisis de datos. Permite trabajar fácilmente con información estructurada en forma de tablas, similares a las de una hoja de cálculo o una base de datos.

Su estructura principal es el DataFrame, que organiza los datos en filas y columnas, aunque también dispone de la estructura Series, utilizada para datos unidimensionales.

Gracias a sus operaciones vectorizadas, permite procesar grandes volúmenes de datos de manera rápida y eficiente.

¿Para qué sirve?

Leer datos desde archivos como CSV, Excel, JSON y bases de datos.

Organizar la información en DataFrames.

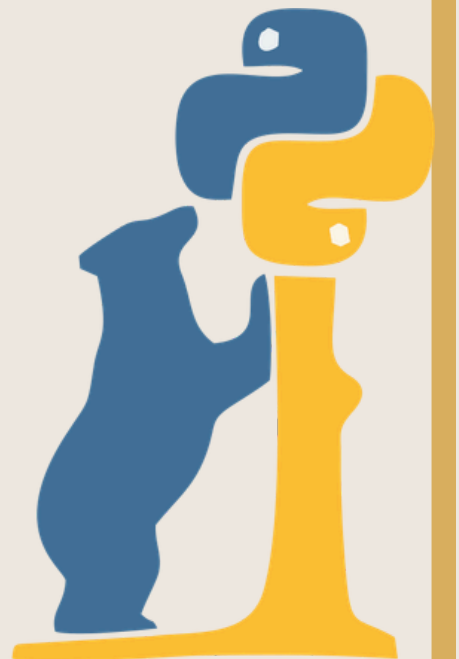
Limpiar datos eliminando o reemplazando valores nulos y corrigiendo errores.

Realizar cálculos estadísticos como promedios, sumas, máximos y mínimos.

Preparar los datos para su análisis, visualización o para modelos de aprendizaje automático (Machine Learning).

Pandas sirve para leer, organizar, limpiar, analizar y manipular datos de manera eficiente en Python. Es una herramienta muy utilizada en análisis de datos, ciencia de datos e inteligencia artificial, ya que facilita el trabajo con información almacenada en tablas.

Pandas



ESTRUCTURA DE DATOS PRINCIPALES

SERIES

1

Arreglo unidimensional etiquetado, similar a una columna en una hoja de cálculo o una tabla SQL.

DATAFRAME

2

Estructura bidimensional, compuesta por filas y columnas. Es una estructura de datos principal en Pandas, muy similar a una hoja de cálculo.

RELACIÓN ENTRE SERIES Y DATAFRAME

3

Un n DataFrame está compuesto por múltiples Series, cada columna del DataFrame es una Series
·Las Series comparten el mismo índice (filas).

¿Cómo se utiliza?

Contenido:

- Instalación y Entorno:
 - Google Colab: Viene preinstalado. Solo se importa.
 - Local: pip install pandas
- Importación:

```
```python
import pandas as pd
import numpy as np
```
```
- Creación de un DataFrame desde un diccionario:

```
```python
df = pd.DataFrame({
 'Columna1': [1, 2, 3],
 'Columna2': ['A', 'B', 'C']
})
```
```
- **Métodos clave para inspeccionar datos:**
 - `df.head()` → Muestra las primeras filas.
 - `df.info()` → Muestra el resumen del DataFrame (tipos, valores no nulos).
 - `df.isnull().sum()` → Cuenta los valores nulos por columna.

Su validación

Contenido:

- Detección de valores nulos:

```
```python
print(df.isnull().sum())
```
```

Resultado:

```
```
Vendedor 0
Region 0
Monto_Venta 0
Unidades 0
dtype: int64
```
```

- Imputación (relleno de valores nulos) con la media:

```
```python
media = df['Examen_1'].mean()
df['Examen_1'] = df['Examen_1'].fillna(media)
```
```

- Verificación de tipos de datos:

```
```python
df.info()
```
```


Ejercicio #1

```
print("\n--- RESULTADO DEL ANÁLISIS POR REGIÓN ---")
display(ventas_por_region)
```

--- VALIDACIÓN DE DATOS ---

Valores nulos por columna:

```
Vendedor      0
Region        0
Monto_Venta    0
Unidades       0
dtype: int64
```

Información del DataFrame:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 6 entries, 0 to 5
```

```
Data columns (total 4 columns):
```

| # | Column | Non-Null Count | Dtype |
|---|-------------|----------------|---------|
| 0 | Vendedor | 6 non-null | object |
| 1 | Region | 6 non-null | object |
| 2 | Monto_Venta | 6 non-null | float64 |
| 3 | Unidades | 6 non-null | int64 |

```
dtypes: float64(1), int64(1), object(2)
```

```
memory usage: 324.0+ bytes
```

--- RESULTADO DEL ANÁLISIS POR REGIÓN ---

| | Region | Total_Ventas | Promedio_Venta | Total_Unidades |
|---|--------|--------------|----------------|----------------|
| 1 | Oeste | 3900.75 | 1950.375 | 29 |
| 0 | Norte | 2700.50 | 1350.250 | 22 |
| 2 | Sur | 1800.25 | 900.125 | 13 |

En este ejemplo se crea un DataFrame con información de ventas, que incluye el vendedor, la región, el monto de venta y las unidades vendidas. Primero, el programa valida que no existan valores nulos y verifica los tipos de datos para asegurar que la información sea correcta. Luego, utiliza la función `groupby()` para agrupar las ventas por región y calcula el total de ventas, el promedio de ventas y el total de unidades vendidas mediante `agg()`. Finalmente, los resultados se ordenan de mayor a menor según el total de ventas, permitiendo identificar fácilmente cuál fue la región con mejor desempeño.

Ejercicio #2

Este ejemplo se muestra como se facilita la limpieza y el procesamiento de datos.

Primero, se crea un DataFrame con información de estudiantes que contiene valores faltantes.

Luego, esos datos incompletos se reemplazan utilizando la media de cada examen mediante `fillna()`, evitando la pérdida de información.

Después, se calcula el promedio final de cada estudiante y, con `np.where()`, se clasifica automáticamente su estado como "Aprobado" o "Reprobado" según si el promedio es mayor o igual a 70. Este proceso demuestra cómo Pandas permite transformar datos incompletos en información útil para realizar análisis de manera rápida, organizada y eficiente.

```
'Examen_1': [85.0, np.nan, 90.0, 60.0, np.nan, 75.0],
'Examen_2': [88.0, 92.0, np.nan, 65.0, 70.0, 80.0],
'Asistencia_Pct': [90, 95, 80, 70, 85, 90]
}

df_estudiantes = pd.DataFrame(data_estudiantes)
print("--- DATOS ORIGINALES CON VALORES NULOS ---")
display(df_estudiantes)

# 2. Imputación de valores faltantes con el promedio de cada examen
media_e1 = df_estudiantes['Examen_1'].mean()
media_e2 = df_estudiantes['Examen_2'].mean()

df_estudiantes['Examen_1'] = df_estudiantes['Examen_1'].fillna(media_e1)
df_estudiantes['Examen_2'] = df_estudiantes['Examen_2'].fillna(media_e2)

# 3. Cálculo del promedio final y validación de aprobación (>= 70)
df_estudiantes['Promedio_Final'] = (df_estudiantes['Examen_1'] + df_estudiantes['Examen_2']) / 2

# Condicional con np.where (Ciclo optimizado vectorizado)
df_estudiantes['Estado'] = np.where(df_estudiantes['Promedio_Final'] >= 70, 'Aprobado', 'Reprobado')

print("\n--- DATOS PROCESADOS Y LIMPIOS ---")
display(df_estudiantes)
```

--- DATOS ORIGINALES CON VALORES NULOS ---

| | Estudiante | Examen_1 | Examen_2 | Asistencia_Pct |
|---|------------|----------|----------|----------------|
| 0 | Luis | 85.0 | 88.0 | 90 |
| 1 | María | NaN | 92.0 | 95 |
| 2 | Jorge | 90.0 | NaN | 80 |
| 3 | Carmen | 60.0 | 65.0 | 70 |
| 4 | Paola | NaN | 70.0 | 85 |
| 5 | Andrés | 75.0 | 80.0 | 90 |

--- DATOS PROCESADOS Y LIMPIOS ---

| | Estudiante | Examen_1 | Examen_2 | Asistencia_Pct | Promedio_Final | Estado |
|---|------------|----------|----------|----------------|----------------|-----------|
| 0 | Luis | 85.0 | 88.0 | 90 | 86.50 | Aprobado |
| 1 | María | 77.5 | 92.0 | 95 | 84.75 | Aprobado |
| 2 | Jorge | 90.0 | 79.0 | 80 | 84.50 | Aprobado |
| 3 | Carmen | 60.0 | 65.0 | 70 | 62.50 | Reprobado |
| 4 | Paola | 77.5 | 70.0 | 85 | 73.75 | Aprobado |
| 5 | Andrés | 75.0 | 80.0 | 90 | 77.50 | Aprobado |

Conclusiones

Resultados de la Investigación

- Pandas es una herramienta poderosa y esencial para el análisis de datos en Python.
- Permite realizar tareas complejas de limpieza, transformación y análisis con código mínimo y eficiente.
- El uso de operaciones vectorizadas (como groupby, fillna, np.where) elimina la necesidad de bucles for tradicionales, mejorando drásticamente el rendimiento, especialmente con grandes volúmenes de datos.
- La capacidad de manejar datos incompletos y realizar validaciones robustas hace que Pandas sea ideal para entornos del mundo real, como el análisis de ventas o la evaluación académica.



Muchas
GRACIAS